

THE CLAUDE BEAST SERIES

The Claude Beast

Claude Sub Agents

Spawning specialized AI workers for parallel, focused execution

Sub-agents are isolated AI workers spawned via the Task tool

- Specialized AI assistants within Claude Code for specific tasks
- Each gets its own context window, system prompt, and tool access
- Spawned via the Task tool with configurable model selection
- Results are summarized and returned to the main conversation
- Flat architecture: sub-agents cannot spawn other sub-agents
- Core benefit: context isolation prevents "context pollution"

Key Insight

Context isolation is the key innovation -- each sub-agent starts fresh, keeping the main conversation clean and focused.

Three core agents cover the most common delegation patterns

EXPLORE AGENT

- Fast, read-only codebase navigation
- Model: Haiku (low-latency)
- Tools: Read-only (Write/Edit denied)
- Thoroughness: quick / medium / very thorough

PLAN & GENERAL-PURPOSE

- Plan: Research agent for plan mode
- Read-only, inherits main model
- General-Purpose: Complex multi-step tasks
- All tools available, inherits main model
- Use when: exploration + modification needed

Define your own agents as Markdown files with YAML frontmatter

COMMON AGENT PATTERNS

- **code-reviewer** — Code quality analysis and PR reviews
- **security-auditor** — Vulnerability detection, OWASP checks
- **test-runner** — Test execution and coverage analysis
- **debugger** — Root cause analysis and bug investigation
- **doc-writer** — Documentation generation

AGENT SCOPES (by priority)

- 1. --agents CLI flag (session only)
- 2. .claude/agents/ (project scope)
- 3. ~/.claude/agents/ (user scope)
- 4. Plugin agents/ directory
- Key frontmatter: name, description, tools, model, permissionMode, hooks, memory, isolation, maxTurns

Start a Claude session with your agent

* Claude Code

SCROLL: 0/30

vsCode/startapp/beast

> `claude --agent code-fixer` 

✱ dkod



Claude Code v2.1.59

Opus 4.6 · Claude Max

@code-fixer · ~/vsCode/startapp/beast

/remote-control is active. Code in CLI or at
https://claude.ai/code/session_


code-fixer

> `Try "write a test for <filepath>"`

Opus 4.6 | [no context yet] | no git | **beast**

› What agent are you? What are your instructions and tool restrictions?

- I am **Claude Code's Code Fixer Agent**, specialized in implementing precise, targeted fixes based on code review feedback (specifically from Greptile code review). Here's a summary:

My Role

I fix issues in **exactly one file** at a time, based on specific review comments provided to me.

Key Instructions

1. Read the assigned file carefully
2. Use **LSP tools** to understand types, definitions, and references before making changes
3. **Analyze** the Greptile review issues provided
4. **Plan** minimal, surgical fixes
5. **Implement** only the fixes needed to address the reported issues
6. **Verify** that TypeScript compiles, linting passes, and tests pass after every edit

Critical Restrictions

What I Can Do:

- Modify **only the single file** assigned to me
- Fix **only the issues** provided in the review input
- Add imports only if strictly necessary for the fix

What I Cannot Do:

- Modify any other files beyond my assigned file

- Refactor unrelated code (even if I notice improvements)
- Add new features or enhancements beyond what's needed for the fix
- Make assumptions if the issue is ambiguous – I must report this instead

Tool Access

I have access to:

- **Bash** – run shell commands (build, lint, test, git, etc.)
- **Glob** – find files by pattern
- **Grep** – search file contents
- **Read** – read files
- **Edit** – make precise string replacements in files
- **Write** – write/overwrite files
- **NotebookEdit** – edit Jupyter notebooks
- **WebFetch / WebSearch** – fetch web content or search the web
- **Skill** – invoke specialized skills
- **Browser automation tools** (mcp__claude-in-chrome__*)

Quality Gate

I **must** run full verification before reporting completion:

```
bun run lint && bunx tsc --noEmit && cd web && bunx tsc --noEmit && cd .. && bun test
```

If any check fails, I must fix it. I only report success when all checks pass.

When I Can't Fix Something

If the fix requires changes to other files, the issue is unclear, or the fix would break functionality, I report this clearly rather than guessing.

In short: I'm a surgical, single-file fixer that only addresses reported issues with verified correctness.

Context isolation is the core architectural innovation

- Each sub-agent gets its own independent context window
- Starts fresh with only system prompt + task description
- Results are summarized and returned to the main conversation
- Verbose output stays in the sub-agent's context (no pollution)
- Auto-compaction at ~95% capacity prevents overflow
- Transcripts persist at `~/.claude/projects/.../subagents/`
- Sub-agents can be resumed to continue with full history

Key Insight

Only distilled results flow back to the parent. All verbose intermediate steps stay in the sub-agent's isolated context.

Foreground blocks the conversation; background frees you to continue

FOREGROUND (Default)

- Blocking: main conversation pauses
- Interactive: permission prompts pass through
- Use when: results needed before continuing
- Use when: task needs user input

BACKGROUND

- Concurrent: runs while you keep working
- Non-interactive: permissions pre-approved
- Ctrl+B to background a running agent
- /tasks to check all running agents
- Auto-resumes main agent on completion

Three dispatch patterns optimize for different task dependencies



PARALLEL DISPATCH

- Independent tasks, no shared state
- Example: 5 Explore agents mapping different parts of a codebase simultaneously
- All conditions: 3+ tasks, no shared state, clear file boundaries



SEQUENTIAL & BACKGROUND

- Sequential: Tasks have dependencies (B needs A)
- Shared files or state (conflict risk)
- Phase 1: Explore > Phase 2: Implement
- Background: Research or analysis tasks
- Results not blocking current work
- Documentation lookups, security audits

Git worktrees enable true parallel editing of the same files

- Each agent gets a separate Git checkout (branch + working dir)
- Worktrees live under `.claude/worktrees/`
- Enable via frontmatter: `isolation: worktree`
- Or via CLI: `claude --worktree feature-auth`
- Clean worktrees are auto-removed; changed ones persist
- Desktop app creates a worktree for every session by default

Key Insight

With worktrees, Agent A can rewrite `src/auth.ts` while Agent B rewrites the same file -- no conflict risk.

Hub-and-spoke delegation vs mesh-topology collaboration

SUB-AGENTS (Hub-and-Spoke)

- Results report back to main agent only
- Main agent manages all coordination
- Lower token cost (summarized results)
- Best for: focused, self-contained tasks
- Built-in, no configuration needed

AGENT TEAMS (Mesh)

- Teammates message each other directly
- Shared task list, self-coordination
- Higher token cost (full instances)
- Best for: collaborative, cross-layer work
- Requires experimental flag to enable

Choosing the right approach: direct work vs sub-agents vs teams

WORK DIRECTLY WHEN:

- Latency matters (quick, targeted change)
- Multiple phases share significant context
- Task needs frequent back-and-forth
- Iterative refinement with user input

USE SUB-AGENTS WHEN:

- Work is self-contained (returns a summary)
- Enforcing tool restrictions / permissions
- Verbose output (tests, logs, docs)
- Preserving main context window
- Independent research in parallel